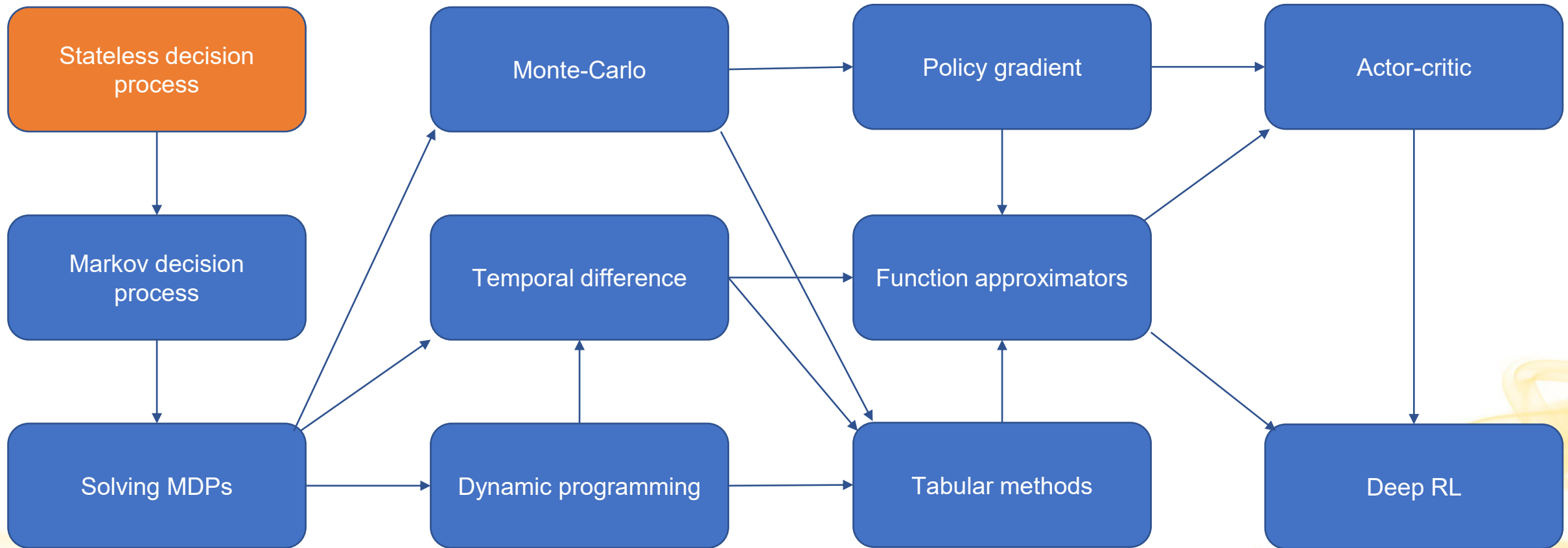


K-ARMED BANDITS

MSAI/MSML 635: Reinforcement Learning

The Roadmap



The Explore / Exploit Dilemma

- Why the “wrong” choice can be the smart choice
- The central tension: exploit what works vs. explore what might be better
- K-armed bandits provide the simplest model of this problem

Learning Objective

- Understand the K-armed bandit problem
- See why pure greedy exploitation can fail
- Learn epsilon-greedy, optimistic initial values, and UCB
- Connect bandits to reinforcement learning and real-world decisions

The Counterintuitive Idea

- Intelligence is not simply choosing the current winner
- Occasional exploration prevents getting trapped in “good enough”
- A small amount of intentional uncertainty can improve long-run decisions

Why Start with a Casino?

- A casino gives a simple, intuitive decision problem
- Actions are easy to visualize as slot-machine pulls
- Rewards provide immediate feedback
- The hidden payout rates create uncertainty



The K-Armed Bandit

- Imagine 3 slot machines and 300 tokens
- Each machine has a different hidden average payout
- The agent does not know which machine is best
- Goal: maximize total reward over the available pulls



Optimal Action



$$\mathbb{E}[a] = -\$0.5$$



$$\mathbb{E}[b] = -\$0.2$$



$$\mathbb{E}[c] = \$0.11$$

Learn the Optimal Action



$$\mathbb{E}[a] = ?$$



$$\mathbb{E}[b] = ?$$



$$\mathbb{E}[c] = ?$$

Learn the Optimal Action



-1, -1, 5

$$\mathbb{E}[a] = 1$$



-0.2, -0.2

$$\mathbb{E}[b] = -0.2$$



-2, -2

$$\mathbb{E}[c] = -2$$

Learn the Optimal Action



-1, -1, 5, -1, -1,
-1, -1, -1, 2, 6,
-1, -1, -1, -1, -1

$$\mathbb{E}[a] = -\$0.5$$



-0.2, -0.2

$$\mathbb{E}[b] = -\$0.2$$



-2, -2

$$\mathbb{E}[c] = \$0.11$$

Notation

k	Number of actions (arms)
t	Discrete time step or play count
$q_*(a)$	True value (expected reward) of action a
$Q_t(a)$	Estimate of $q_*(a)$ at time step t
$N_t(a)$	Number of times action a was selected before step t
R_t	The reward observed at time step t
A_t	The action chosen at time step t



The Core Challenge

- Every token spent is both a decision and a learning opportunity
- The agent must decide how much to use current knowledge
- Exploration can reveal better options; exploitation earns from known good options



Exploitation

- Choose the action currently believed to be best
- Uses existing knowledge to maximize immediate reward
- Example: repeatedly choosing machine #1 because it has paid well so far

Exploration

- Try actions whose value is uncertain
- Exploration improves knowledge of the environment
- A seemingly inferior action may actually be the hidden jackpot

The Exploration–Exploitation Trade-off

- The same token cannot simultaneously explore and exploit
- Exploring can sacrifice immediate reward
- Never exploring can prevent discovery of a better action
- Successful learning requires balancing both

Estimating Action Value

- Estimated Q is based on observed rewards
- Simple sample-average estimate: total observed reward $\div$ number of pulls
- Example: rewards 2, 8, 5 $\rightarrow$ estimate = $15/3 = 5$

Learn the Optimal Action



$$R = \{-1, -1, 5\}$$

$$N_{11}(a) = 3$$

$$q_*(a) = -\$0.5$$

$$Q_{11}(a) = 1$$



$$R = \{-0.2, -0.2\}$$

$$N_{11}(b) = 2$$

$$q_*(b) = -\$0.2$$

$$Q_{11}(b) = -0.2$$



$$R = \{-2, -2\}$$

$$N_{11}(d) = 2$$

$$q_*(d) = \$0.11$$

$$Q_{11}(d) = -2$$

The Greedy Strategy

- Initially sample each machine
- Maintain an estimated value for each action
- After initialization, always select the action with the highest estimate
- This is pure exploitation



Greedy action = Exploit



$$R = \{-1, -1, 5\}$$

$$Q_{11}(a) = 1$$



$$R = \{-0.2, -0.2\}$$

$$Q_{11}(b) = -0.2$$



$$R = \{-2, -2\}$$

$$Q_{11}(d) = -2$$

Or Explore



$$R = \{-1, -1, 5\}$$

$$Q_{11}(a) = 1$$



$$R = \{-0.2, -0.2\}$$

$$Q_{11}(b) = -0.2$$



$$R = \{-2, -2\}$$

$$Q_{11}(d) = -2$$

Solution 1: Epsilon-Greedy

- Be greedy most of the time
- Occasionally choose an action at random
- The small probability of exploration creates an escape route from local mistakes
- If each action is sampled infinitely, then $Q_t(a)$ will converge on $q_*(a)$
 - Assuming Q is unbiased approximation of q_*

```
epsilon = 0.05
def get_action():
    if random.random() > epsilon:
        return argmaxa(Q(a))
    else:
        return random.choice(A)
```

Example

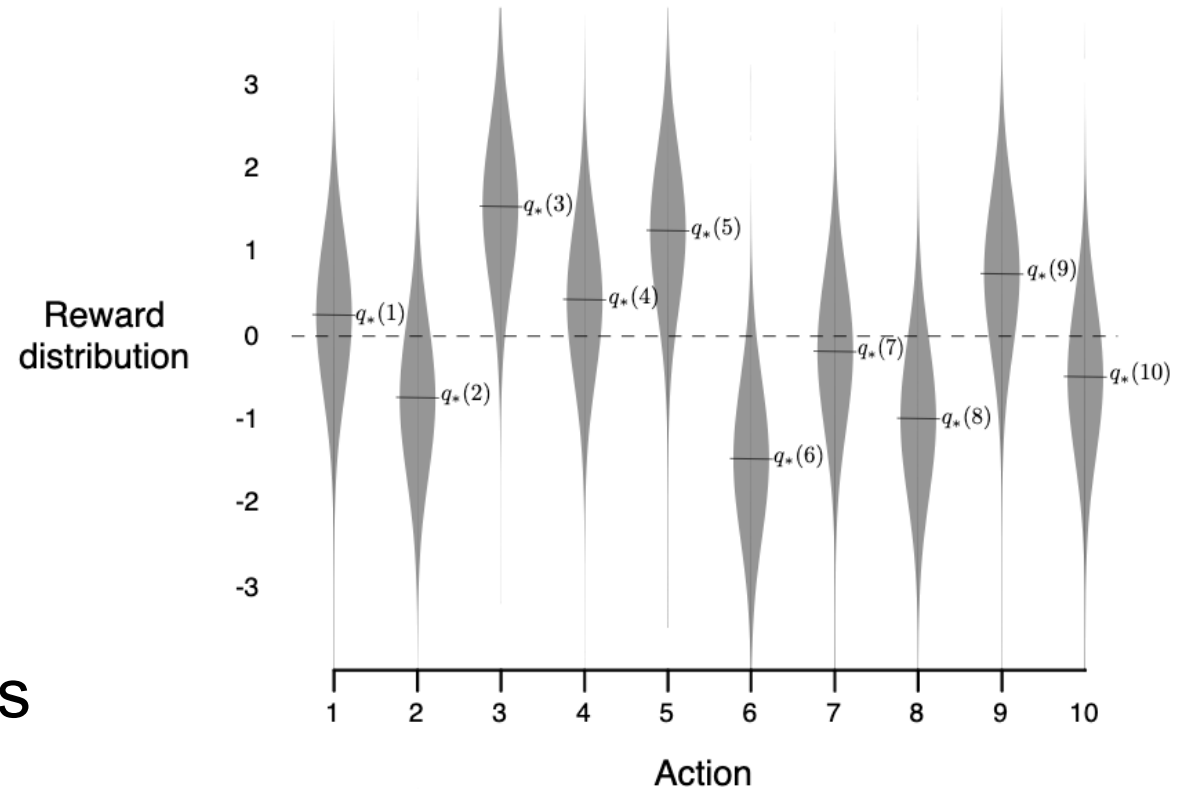
THE SETUP

Truth: $q^*(1)=2$, $q^*(2)=6$, $q^*(3)=4$ | Agent: $\epsilon=0.1$

Step	Decision Logic	Action	Reward	Agent's Brain: N(a), Q(a)
Start				N=(0,0,0), Q=(0,0,0)
1	Explore (Q's are 0)	Arm 3	R=5	N=(0,0,1), Q=(0, 0, 5)
2	EXPLOIT (90%)	Arm 3	R=3	N=(0,0,2), Q=(0, 0, 4)
3	EXPLOIT (90%)	Arm 3	R=4	N=(0,0,3), Q=(0, 0, 4)
4	EXPLORE (10%)	Arm 1	R=1	N=(1,0,3), Q=(1, 0, 4)
5	EXPLOIT (90%)	Arm 3	R=4	N=(1,0,4), Q=(1, 0, 4)
6	EXPLORE (10%)	Arm 2	R=8	N=(1,1,4), Q=(1, 8, 4)
7	EXPLOIT (argmax is now 2!)	Arm 2	R=5	N=(1,2,4), Q=(1, 6.5, 4)

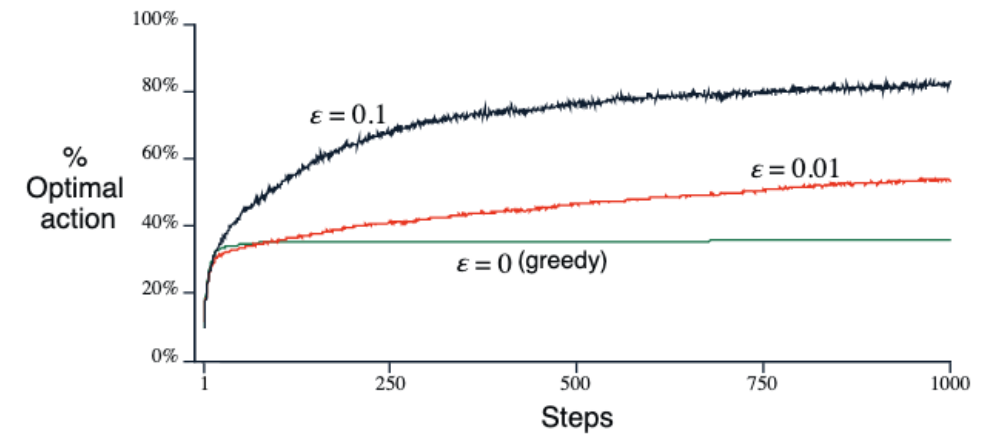
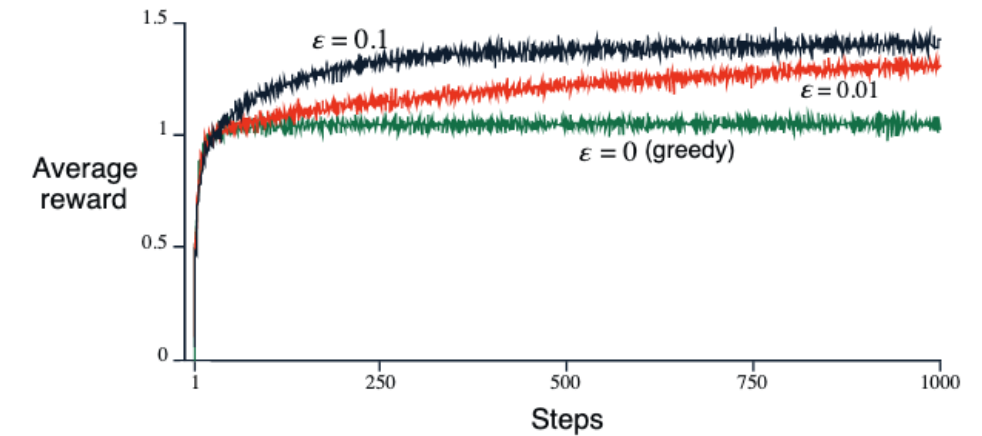
Example

- $k = 10$
- $A = \{1, \dots, 10\}$
- $\Pr\{r|a\} \sim \mathcal{N}(q_*(a), 1)$
- The optimal policy is to always choose a_3
- But we don't know that initially



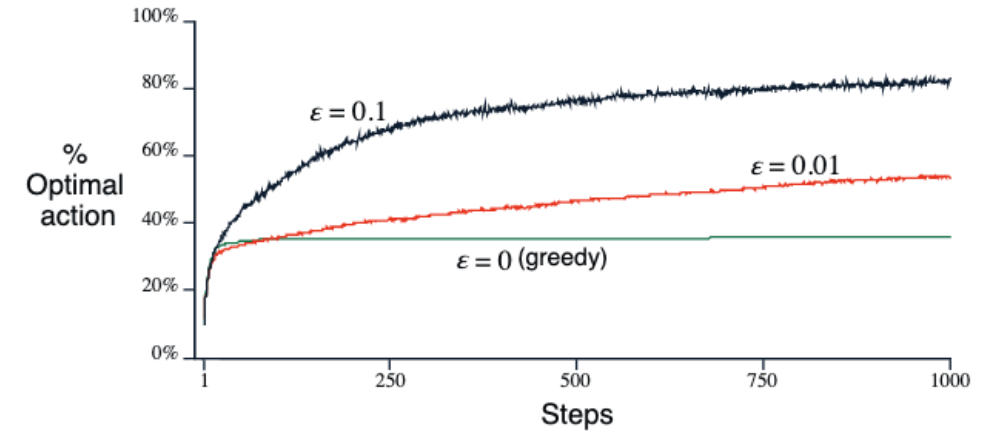
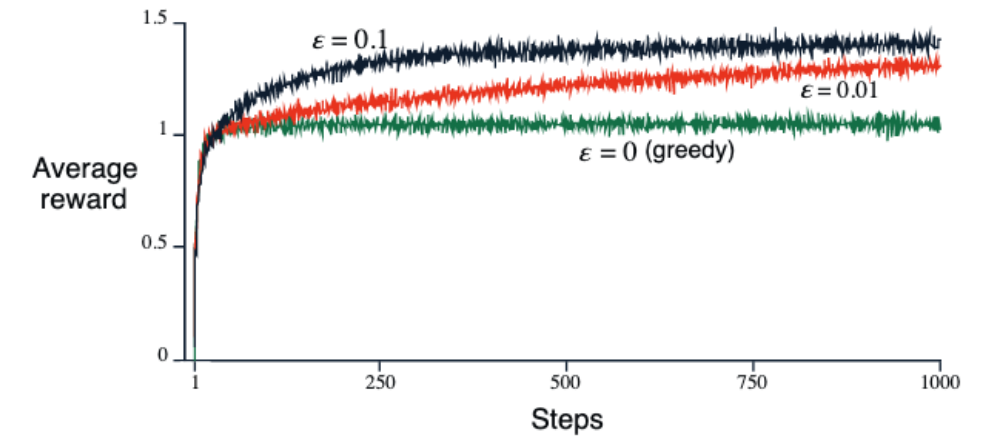
Results

- Average performance of ϵ -greedy action-value methods on the 10-armed testbed
 - $\epsilon = 0$
 - $\epsilon = 0.1$
 - $\epsilon = 0.01$
- Averages over 2000 runs each with 1000 steps
- All methods used sample averages as their action-value estimates



Results

- Average performance of ϵ -greedy action-value methods on the 10-armed testbed
 - $\epsilon = 0$
 - $\epsilon = 0.1$
 - $\epsilon = 0.01$
- Averages over 2000 runs each with 1000 steps
- All methods used sample averages as their action-value estimates



Incremental Sample Averages

- Sample averages: $Q_{t+1}(a) = \frac{\sum_{i=1}^t r_i}{N_t(a)}$
- Memory and computational inefficient, $O(N_t(a))$
- Temporal difference:

It does not need to recompute the complete average from scratch

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a)} [r_t - Q_t(a)]$$

- More generally:
 - $NewEstimate \leftarrow OldEstimate + w_t [Target - OldEstimate]$

Nonstationary Outcomes

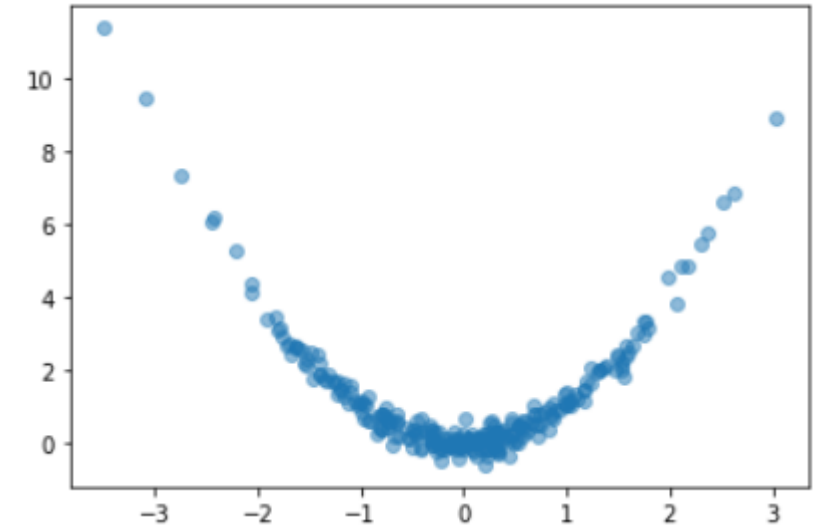
- Averaging over samples assumes stationary distribution of outcomes
- What if the reward probabilities change over time?
 - Give more weight to recent observations!

$$\begin{aligned} \text{constant} \\ \bullet \quad Q_{t+1}(a) &= Q_t(a) + \alpha [r_t - Q_t(a)] \quad 0 < \alpha < 1 \\ &= \alpha r_t + (1 - \alpha)\alpha r_{t-1} + (1 - \alpha)^2 \alpha r_{t-2} + \cdots + (1 - \alpha)^{t-1} \alpha r_1 + (1 - \alpha)^t Q_1(a) \end{aligned}$$

- a.k.a., Exponential Decay or Exponentially Weighted Moving Average

Setting the step size α

- The learning rate cannot be too large
 - Won't converge
- Or too small
 - Premature convergence
- Good practice: Start large and decrease
 - E.g., $\alpha_t(a) = \frac{1}{N_t(a)}$ (results in sample averages)
- Conditions that guarantee convergence to optimum (global if assuming convexity)
 - $\sum_t \alpha_t(a) = \infty$
 - $\sum_t \alpha_t^2(a) < \infty$

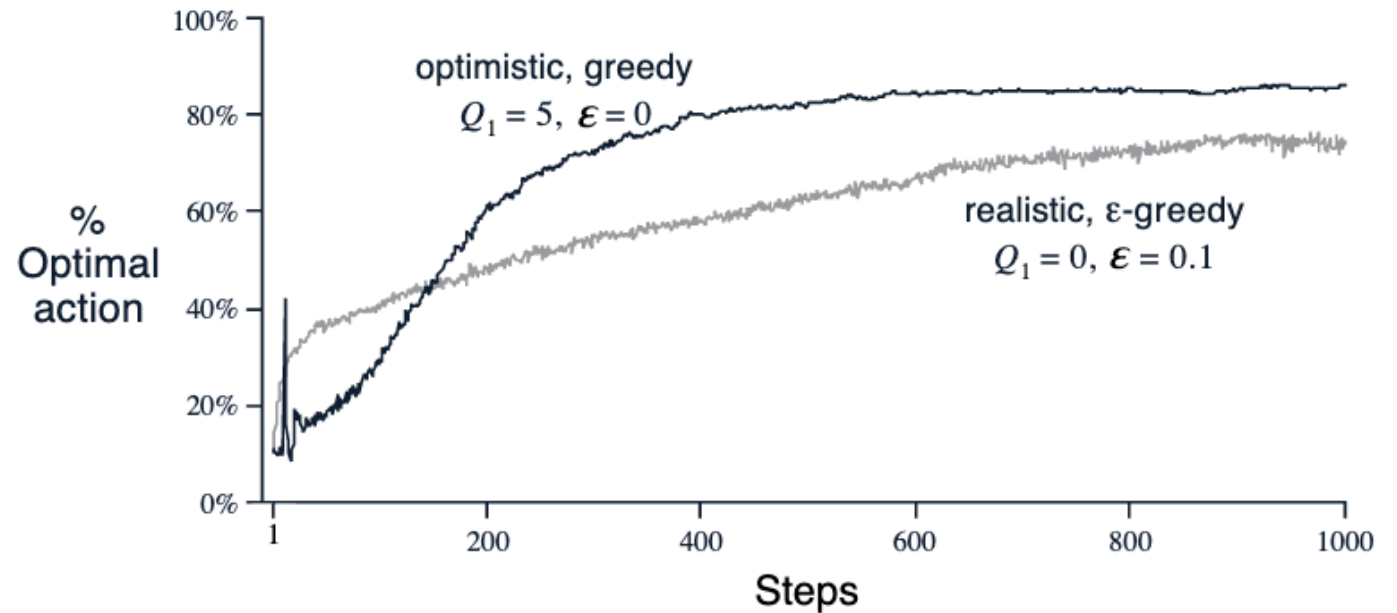


Initial Q values + greedy

- $Q_{t+1}(a) = \begin{cases} Q_t(a) + \alpha_t[r_t - Q_t(a)] , & N_t(a) > 0 \\ ? , & N_t(a) = 0 \end{cases}$
- What would happen if we set the default value to the minimal achievable outcome?
 - Initial values introduce bias
- What would happen if we set the default value to the maximal achievable outcome?
 - Initial values can encourage exploration!

10-bandits problem

- $k = 10$
- $A = \{1, \dots, 10\}$
- $\Pr\{r|a\} \sim \mathcal{N}(q_*(a), 1)$
- $\max[q_*(a)] = 3$
- $\alpha = 0.1$
- Setting initial value to 5 and then following greedy selection is more effective than ε -greedy



Limitations of Epsilon-Greedy

- Exploration is completely random
- The agent may explore an arm it already knows well
- The same exploration probability is used throughout the run
- This motivates smarter exploration strategies

Solution 2: Optimistic Initial Q values

- Powerful for stationary outcomes
- Start with artificially high initial Q values
- Example: maximum possible reward = 10, but initialize every Q to 20
- Unlikely to help with the general nonstationary case. Why?
 - Encourages exploration at early stages
 - Exploration incentive decays over time

Why Optimism Creates Exploration

- All actions initially look excellent
- After trying an action, its estimate drops toward reality
- Untested actions retain their optimistic values
- Greed therefore drives the agent toward untried actions

Optimistic Values: Three-Arm Example

- Initial values: $Q_1=Q_2=Q_3=20$
- Try arm 1 $\rightarrow$ reward 3 $\rightarrow$ Q_1 becomes 3
- Arm 2 and arm 3 still look like 20
- Greedy selection is forced toward an untried arm
 - No random exploration mechanism is required



Optimistic Values: Three-Arm Example

THE SETUP

Truth: $q^*=(2,6,4)$ | Agent: Greedy $\epsilon=0$ | Initial Brain: $Q=(20,20,20)$

Pull 1: Picks Arm 1 (tie-break). Reward = 3.

Agent's Brain: $Q = (3, 20, 20)$

Arm 1 was a disappointment.

Pull 2: Greed forces it to pick Arm 2. Reward = 8.

Agent's Brain: $Q = (3, 8, 20)$

Arm 2 was also a disappointment.

Pull 3: Greed forces it to pick the last untried arm: Arm 3.

Agent's Brain: $Q = (3, 8, \dots)$

Epsilon-Greedy vs. Optimistic Initial Values

- Epsilon-greedy: random exploration throughout
- Optimism: strong early exploration driven by high initial estimates
- Epsilon-greedy is persistent; optimism's exploration naturally fades



Limitation of Optimistic Initial Values

- After enough trials, Q estimates settle near actual values
- The initial optimism loses its exploratory influence
- If the environment changes, the agent may adapt slowly



Solution 3: Upper Confidence Bound (UCB)

- UCB explicitly combines estimated value and uncertainty
- Instead of random exploration, it explores with purpose
- It asks which action has the strongest optimistic potential

The UCB Idea

- Choose the arm with the highest estimated reward plus an uncertainty bonus
- Good estimates favor exploitation
- High uncertainty creates curiosity
- The method balances both at every step

Upper-Confidence-Bounds Action Selection

- $A_{t+1} = \operatorname{argmax}_a \left[\overset{\substack{\text{current} \\ \text{estimated value}}}{Q_t(a)} + c \sqrt{\frac{\overset{\substack{\text{\# of pulls}}{\ln(t)}}{\underset{\substack{\text{\# of pulls} \\ \text{of action A}}}{N_t(a)}}}} \right]$
- $c > 0$ is a parameter for controlling the degree of exploration
- Provides a “bonus” in value estimation for underexplored actions
- Each time a is selected the uncertainty is presumably reduced: $N_t(a)$ increments, and, as it appears in the denominator, the uncertainty term decreases

Upper-Confidence-Bounds Action Selection

- On the other hand, each time an action other than a is selected, t increases but $N_t(a)$ does not; because t appears in the numerator, the uncertainty bonus increases
- The use of the natural logarithm means that the increases get smaller over time, but are unbounded
- All actions will eventually be selected, but actions with lower value estimates, or that have already been selected frequently, will be selected with decreasing frequency over time.

UCB Example: Arm A vs Arm B

- Arm A: estimated value 8, pulled 100 times
- Arm B: estimated value 7, pulled only 2 times
- Greedy chooses A because $8 > 7$
- UCB may choose B because its uncertainty bonus is much larger



UCB Example: Arm A vs Arm B

Arm A

$$UCB(A) = 8 + \sqrt{\frac{\ln(102)}{100}}$$

$$\ln(102) \approx 4.625$$

$$\sqrt{\frac{4.625}{100}} = \sqrt{0.04625} \approx 0.215$$

$$UCB(A) = 8 + 0.215 = 8.215$$

Arm B

$$UCB(B) = 7 + \sqrt{\frac{\ln(102)}{2}}$$

$$= 7 + \sqrt{\frac{4.625}{2}}$$

$$= 7 + \sqrt{2.3125}$$

$$\approx 7 + 1.521$$

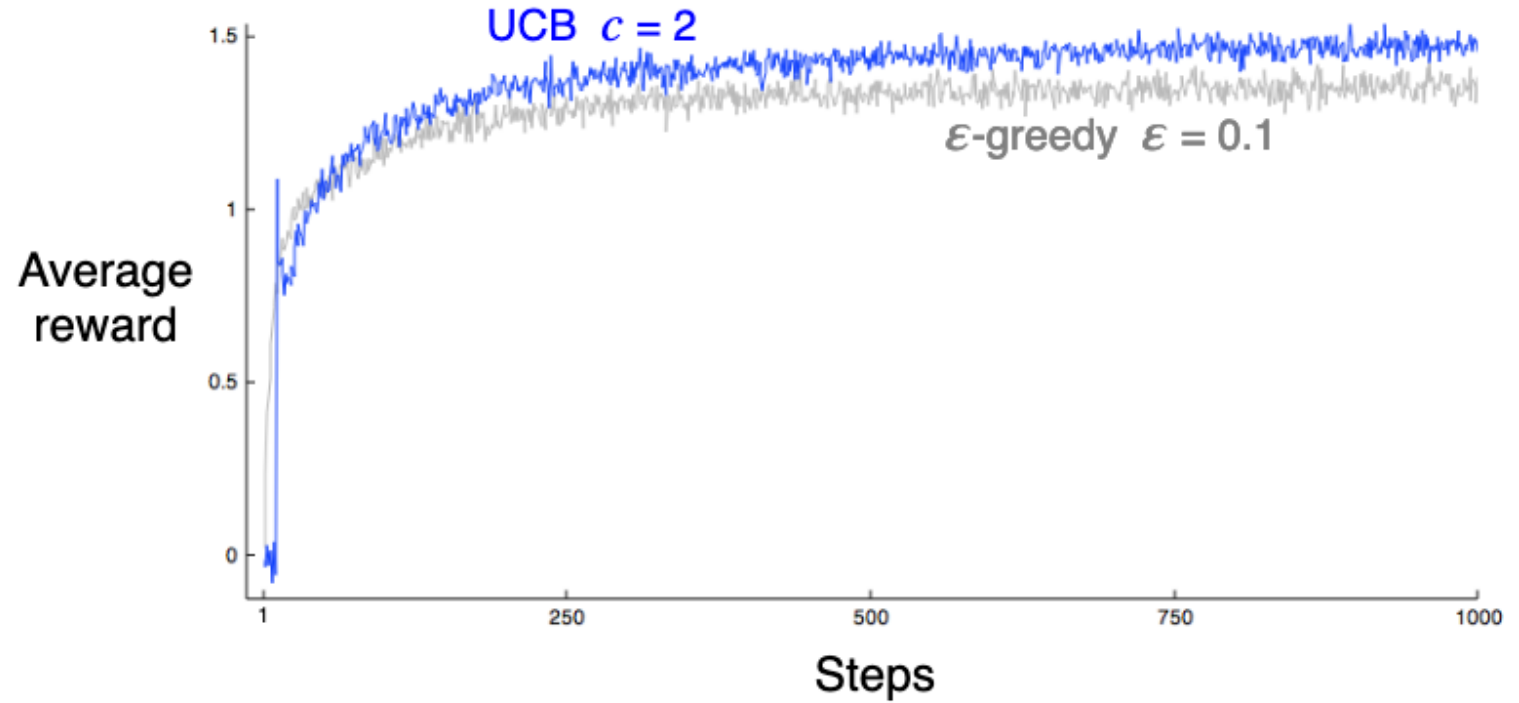
$$UCB(B) = 8.521$$

Why UCB Is Directed Exploration

- UCB does not need a random exploration trigger
- It identifies underexplored actions with high optimistic potential
- Exploration is deterministic given the estimates and parameters

10-bandits problem

- $k = 10$
- $A = \{1, \dots, 10\}$
- $\Pr\{r|a\} \sim \mathcal{N}(q_*(a), 1)$
- $\max[q_*(a)] = 3$
- $\alpha = 0.1$



Comparing the Three Strategies

- Greedy: exploit only; can get trapped
- Epsilon-greedy: random exploration; simple and robust
- Optimistic initialization: directed early exploration
- UCB: value + uncertainty; strategic exploration

From Naive to Strategic

- Greedy agent: commits too quickly
- Epsilon-greedy: adds persistent randomness
- Optimistic initialization: uses optimism to drive early search
- UCB: explicitly reasons about uncertainty

Why Bandits Matter

- The casino is a metaphor for decisions under incomplete information
- The same explore–exploit tension appears beyond slot machines
- Bandit methods turn this trade-off into concrete decision rules

The Limitation of Bandits

- In the basic bandit, the best action does not depend on context
- The machine's true value is assumed to remain fixed
- Real-world decisions often depend on the current situation

Bridge to Full Reinforcement Learning

- In full RL, the best action can depend on the state
- Self-driving: brake or accelerate depends on traffic conditions
- Chess: the best move depends on the board position
- The agent must learn the best action for different states

Policy-based algorithms

- Forget about action-value (Q) estimates, we don't really care about them
- We care about what actions to choose
 - Let's assign a preference to each action and tweak its value
- Define $H_t(a)$ as a numerical score associated with action a
- Which action is selected?
 - $A_t = \underset{a}{\operatorname{argmax}}[H_t(a)]$
 - Hardmax results in no exploration -- deterministic action selection
 - Softmax!

Softmax function

- **Input:** vector of scores
- **Output:** vector of probabilities forming a valid distribution

- $$P\{a_t = a\} = \frac{e^{H_t(a)}}{\sum_{a' \in A} e^{H_t(a')}} = P(a)$$

- $$H_t \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{bmatrix} 6 \\ 9 \\ 2 \end{bmatrix} \Rightarrow \text{softmax} \begin{pmatrix} 6 \\ 9 \\ 2 \end{pmatrix} = \begin{bmatrix} 0.047 \\ 0.952 \\ 0.00087 \end{bmatrix}$$

- That is, with $P(0.95)$ choose a_2 , $P(0.05)$ choose a_1 , and $< P(0.01)$ choose a_3

Softmax function

- Exploration – checked!
- Softmax provides another important attribute – **a differentiable policy**
- Say that we learn that action a_1 results in good relative performance
- Hardmax: $A_t = \underset{a}{\operatorname{argmax}}[H_t(a_1), H_t(a_2), H_t(a_3)]$
 - Change $H(a_1)$ such that $P(a_1)$ is increased, $\frac{\partial P(a_1)}{\partial H(a_1)} = NA$
- Softmax: $P(a_1) = \frac{e^{H_t(a_1)}}{\sum_{a' \in A} e^{H_t(a')}}$
 - Change $H(a_1)$ such that $P(a_1)$ is increased $\rightarrow$ update towards $\frac{\partial P(a_1)}{\partial H(a_1)}$

Baseline, $\overline{R}_t$?

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \overline{R}_t)(1 - P(A_t))$$
$$\forall a \neq A_t, H_{t+1}(a) = H_t(a) - \alpha(R_t - \overline{R}_t)P(a)$$

- If observed reward is bigger than baseline:
 - Increase the score of the chosen action, A_t
 - Decrease the score of all other actions, $\forall a \neq A_t$
- Else do the opposite



$H(a)$	3	2	1
$P(a)$	0.665	0.245	0.09
R	2	8	1

Baseline, $\overline{R}_t$?

*In this example we are updating each machine assuming it was played and the affiliated R_t was observed

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \overline{R}_t)(1 - P(A_t))$$

$$\forall a \neq A_t, H_{t+1}(a) = H_t(a) - \alpha(R_t - \overline{R}_t)P(a)$$



Observed reward

Update direction
for various
baseline values

$H(a)$	3	2	1
$P(a)$	0.665	0.245	0.09
R_t	2	8	1
$(R_t - \overline{R}_t)(1 - P(A_t))$			
$\overline{R}_t = 10$	-2.68	-1.51	-8.19
$\overline{R}_t = 3$	-0.34	3.78	-1.82
$\overline{R}_t = -3$	1.68	8.31	3.64

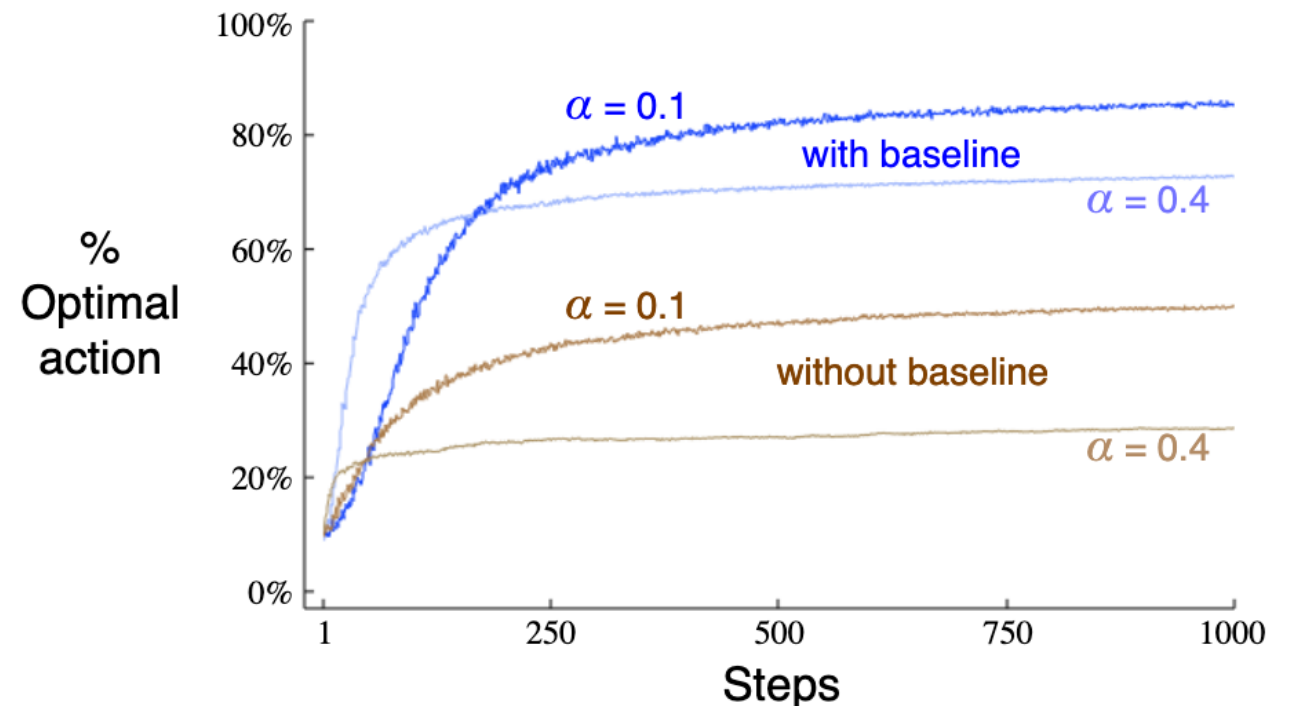
Which baseline value
makes most sense?

Setting $\overline{R}_t = \frac{1}{t} \sum_t r$ is usually a good choice

*More on choosing a baseline later

Baseline Impact

- Average performance of gradient ascend
- With baseline: $\overline{R}_t = \frac{1}{t} \sum_t r$
- No baseline: $\overline{R}_t = 0$
- 10-armed example problem



Overall Results

- A parameter study of the various bandit algorithms
- Each point is the average reward obtained over 1000 steps with a particular algorithm at a particular setting of its parameter

